REAL TIME PATIENT SCHEDULING AND ALERT SYSTEM

TEAM ZYBOTS

240361V – L.H.H.R. Kumar

240470E – K.A.R. Nishaya

240553L – W.B.Y. Ranaweera

240558G – J.L.E. Rathnapala

240710R – W.W.N.D. Wickramarathna

1. INTRODUCTION (Updated with OOSD Perspective)

1.1 Background

Traditional hospital appointment systems rely on fixed time slots. However, consultation durations vary due to emergencies, complx diagnoses, or workflow delays. This leads to

- Long patient waiting times
- Overcrowded waiting areas
- Inefficient doctor time usage
- Increased stress for patients and staff

The **Real Time Patient Scheduling and Alerts System** introduces a **dynamic, object oriented healthcare management platform** that synchronizes patient arrival times with a doctor's real time availability….

1.2 Object-Oriented System Overview

To align with **Object Oriented Software Development (OOSD)**, the system is redesigned around interacting objects...

Core System Objects (Entities)

Object	**Description**
Patient	Books appointments and receives alerts
Doctor	Conducts consultations and updates status
Appointment	Represents a booking between patient and doctor
Schedule	Maintains doctor availability
Notification	Sends alerts to patients

Object	Description
QueueManager	Calculates real-time queue status
Admin	Manages system configuration
Consultation	Represents a live consultation session
IoTDevice (ESP32)	Detects doctor room readiness

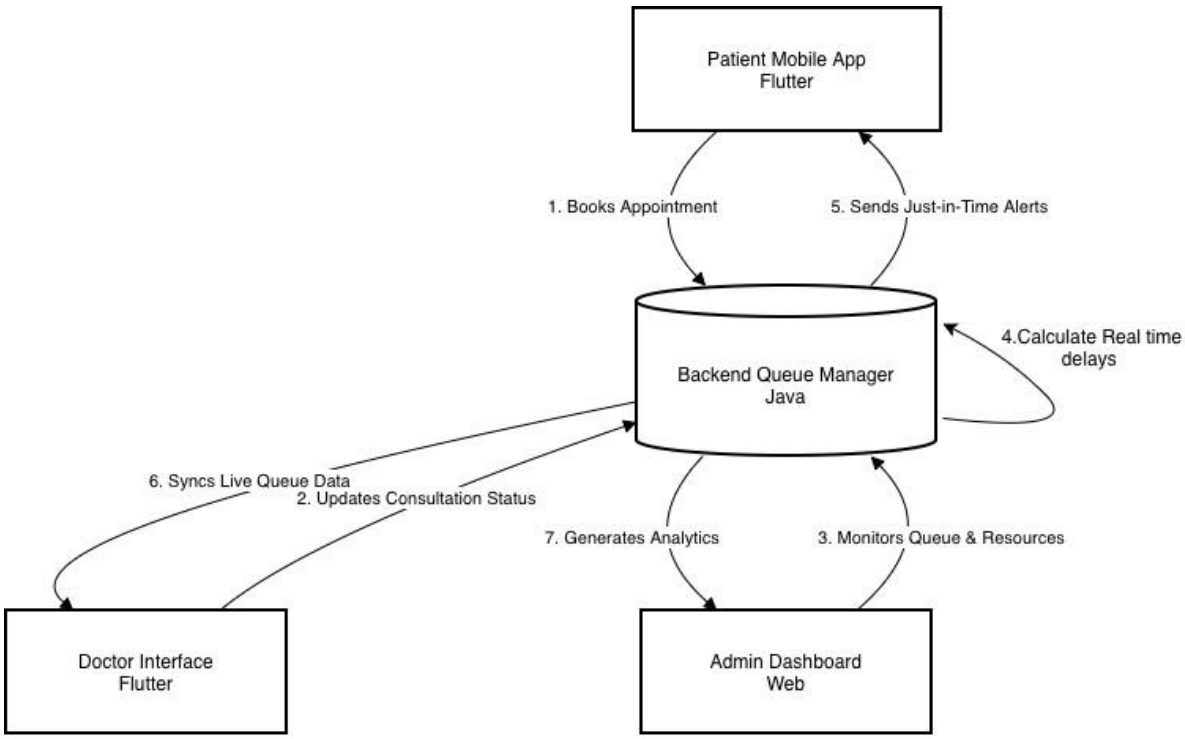
1.3 System Architecture Overview

Layers

1. Presentation Layer
    - Flutter Mobile App (Patient, Doctor, Admin)
2. Application Layer
    - Scheduling Engine
    - Queue Manager
    - Notification Service
3. Data Layer
    - Relational Database (MySQL / PostgreSQL)
    - JPA (Java Persistence API)
    - Hibernate ORM
    - Spring Data JPA
4. Hardware Layer
    - ESP32 Smart Status Controller

- ESP32-CAM

High Level System Architecture Diagram



Value for the User (Patients and Medical Staff)

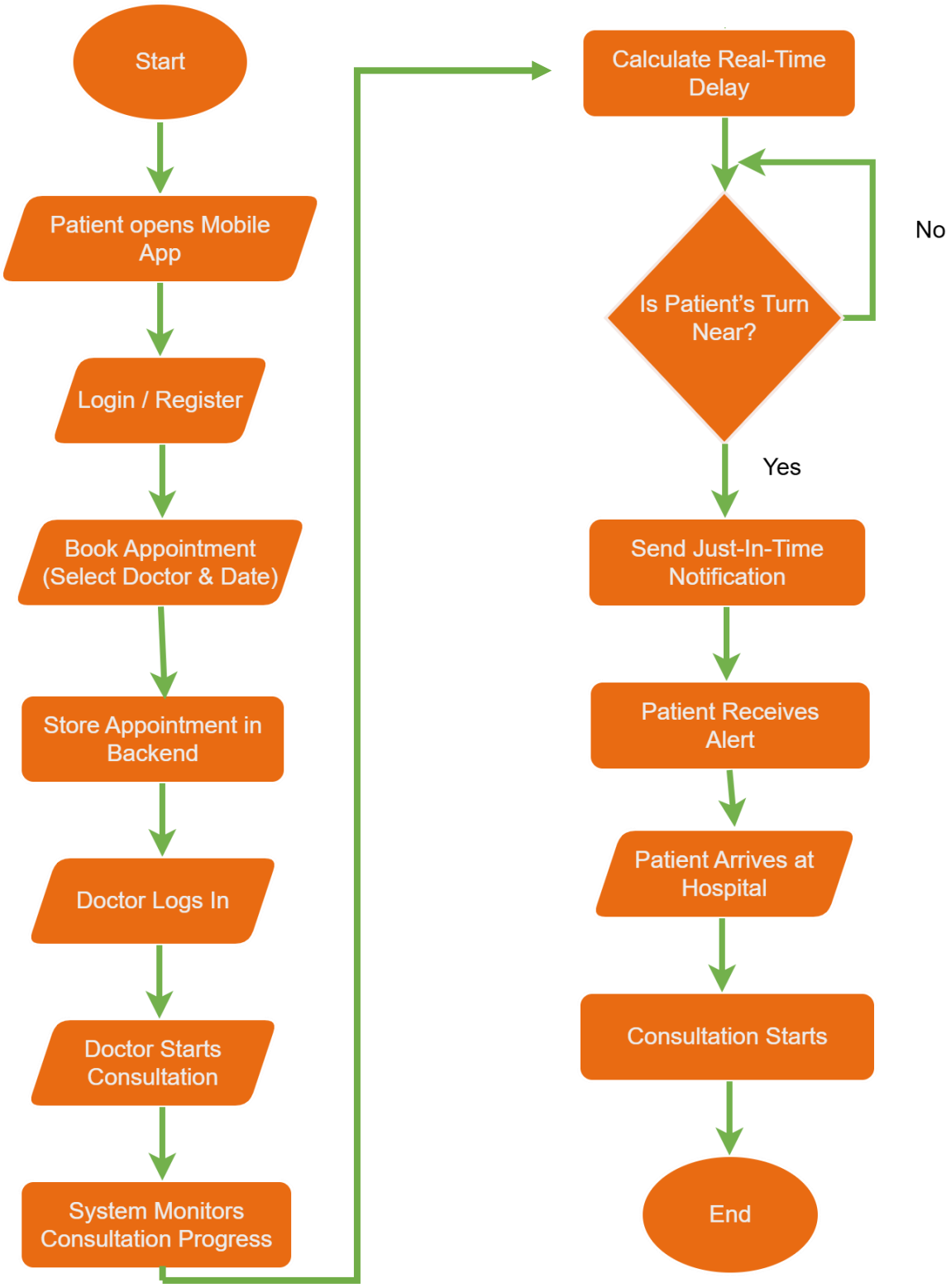
1. Patients:

   Users of the mobile app receive a transparent, stress free experience. Instead of waiting at the clinic, they receive just in time notifications and experience significantly reduced waiting times.

2. Doctors

   Medical professionals can seamlessly view their queue of upcoming patients and manage their consultation flow more efficiently without the pressure of an overcrowded waiting room outside their door.

System Level Flowchart



1.4 Key Features (Customer & User Perspective)

For Patients

- Book and manage appointments
- View real time queue position
- Receive Just in Time alerts
- Reduced waiting time
- Provide feedback

For Doctors

- View daily schedule
- Update consultation status
- Mark emergency delays
- Monitor upcoming patients

For Hospital Administration

- Monitor real time operations
- Manage doctor schedules
- Generate performance reports
- View analytics dashboards

2. NON-FUNCTIONAL REQUIREMENTS

2.1 Essential Non-Functional Requirements

1. Usability

- Simple UI for all user roles
- Clear navigation
- Accessibility support

2. Reliability

- Notifications must be delivered consistently
- Accurate delay prediction

3. Availability

- 24/7 system operation

4. Performance

- Real-time delay calculation < 2 seconds
- Notification latency < 5 seconds

5. Security

- Role-based access control
- Encrypted data storage
- Secure API access

6. Scalability

- Support thousands of patients
- Multi hospital expansion capability

7. Maintainability

- Modular architecture
- Clean OOP design
- Easy integration of ML upgrades

8. Interoperability

- Android & iOS compatibility
- Future integration with hospital ERP

9. Robustness

- Handles network interruptions
- Graceful recovery from failures

10. Traceability

- Audit logs for appointments
- Notification history logs

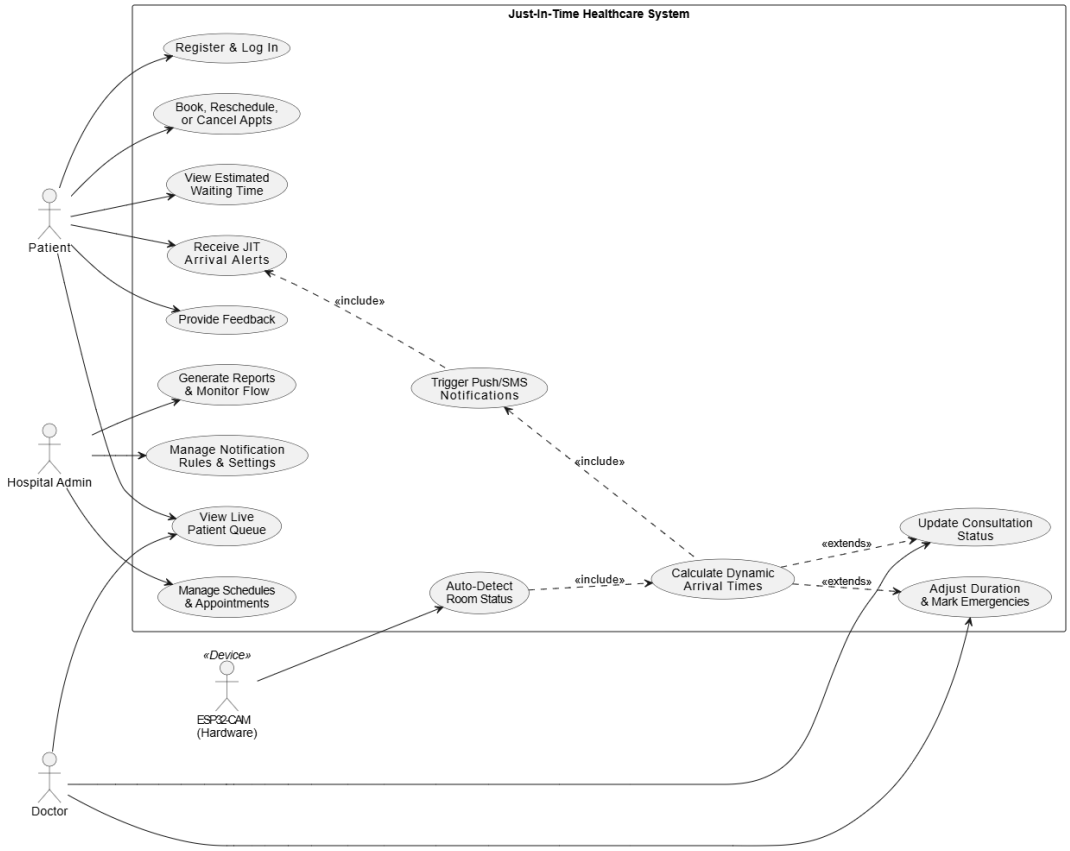
3. ANALYSIS SECTION OOAD APPROACH

This section uses UML behavioral models.

3.1 USE CASE DIAGRAM

Actors

- Patient
- Doctor
- Admin
- IoT Device

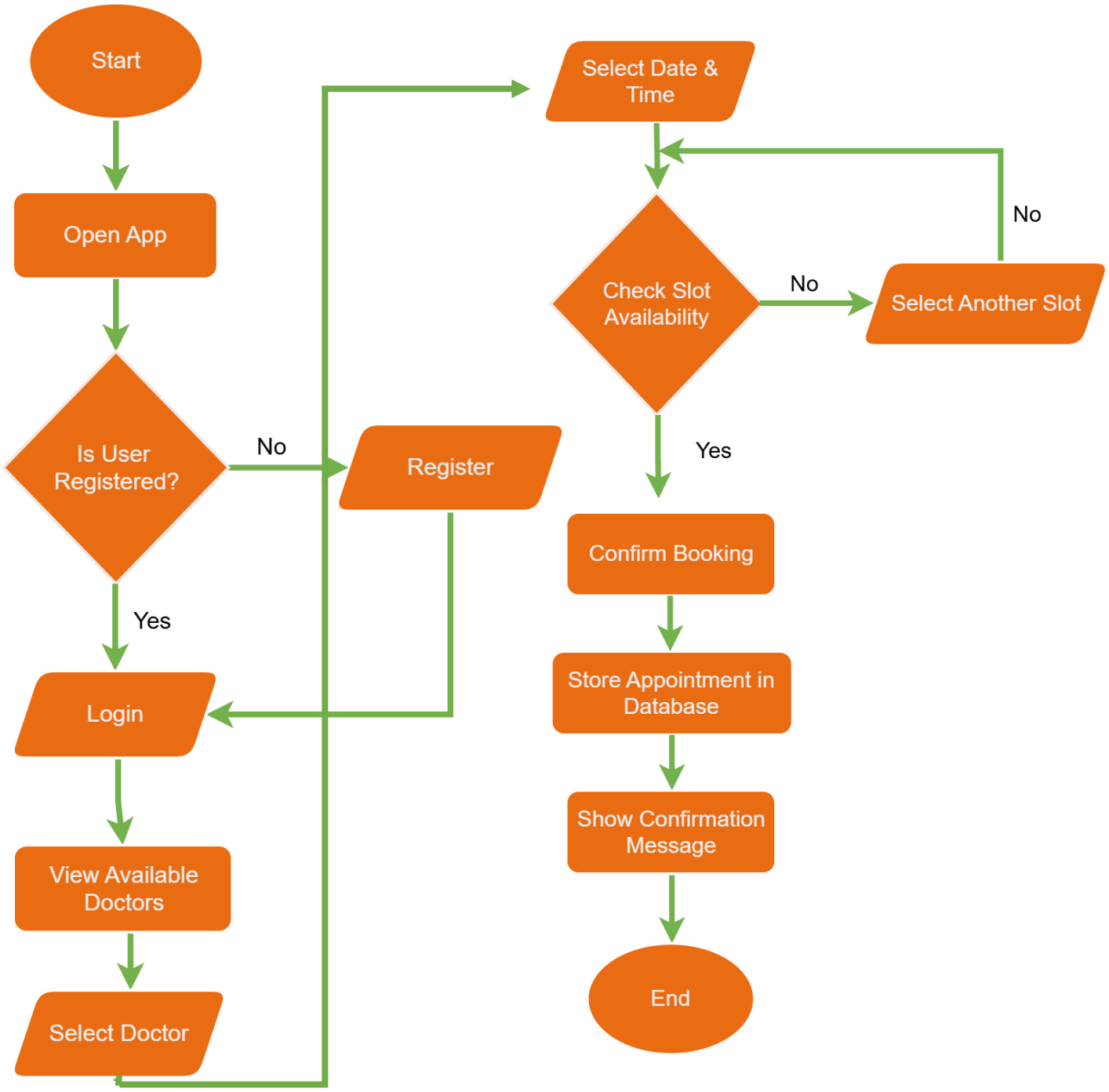


Main Use Cases

Patient

- Register/Login
- Book Appointment
- Cancel Appointment
- View Queue Status
- Receive Notification
- Provide Feedback

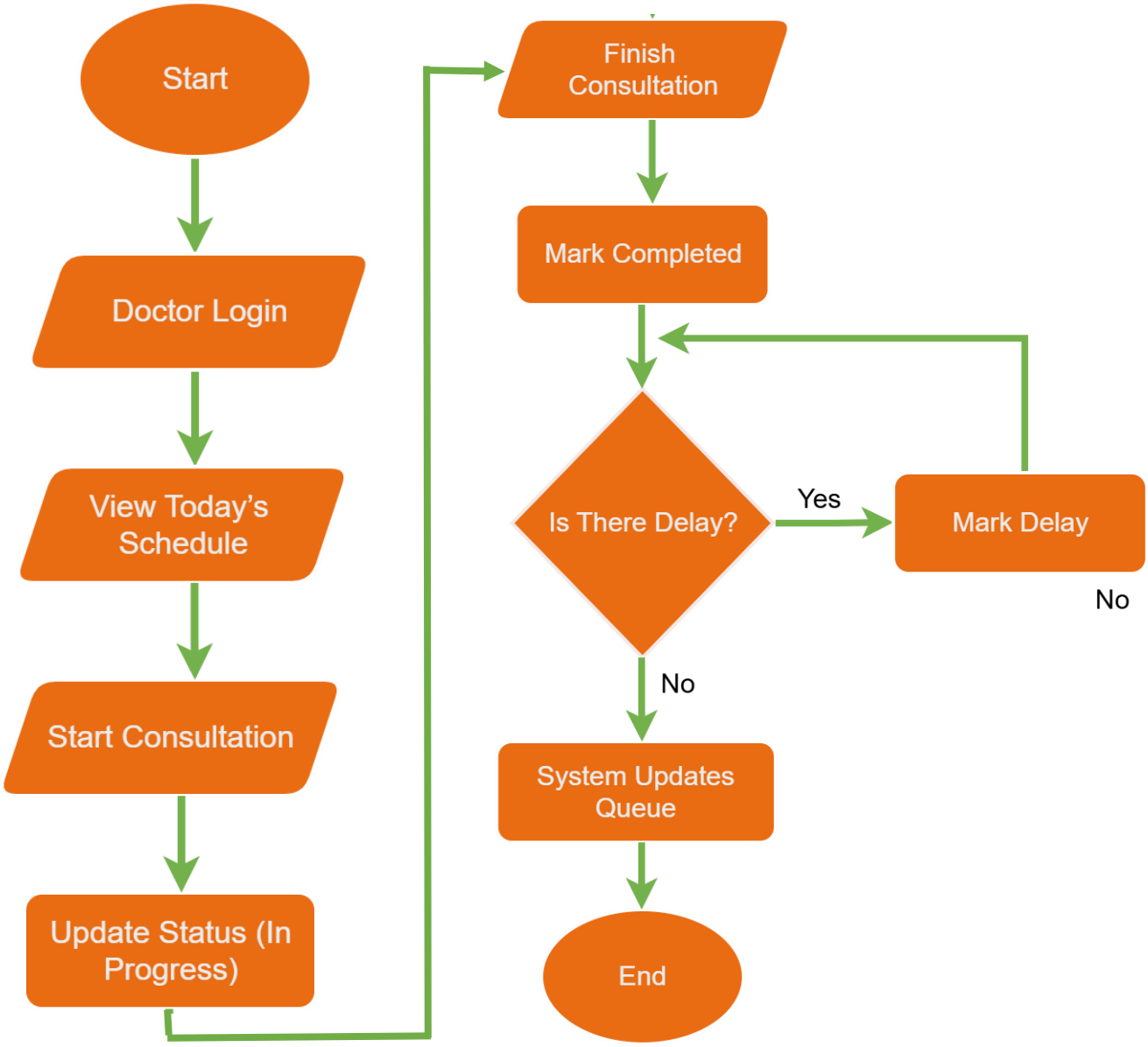
Patient Booking Process



Doctor

- Login
- Update Consultation Status
- View Schedule
- Mark Emergency Delay

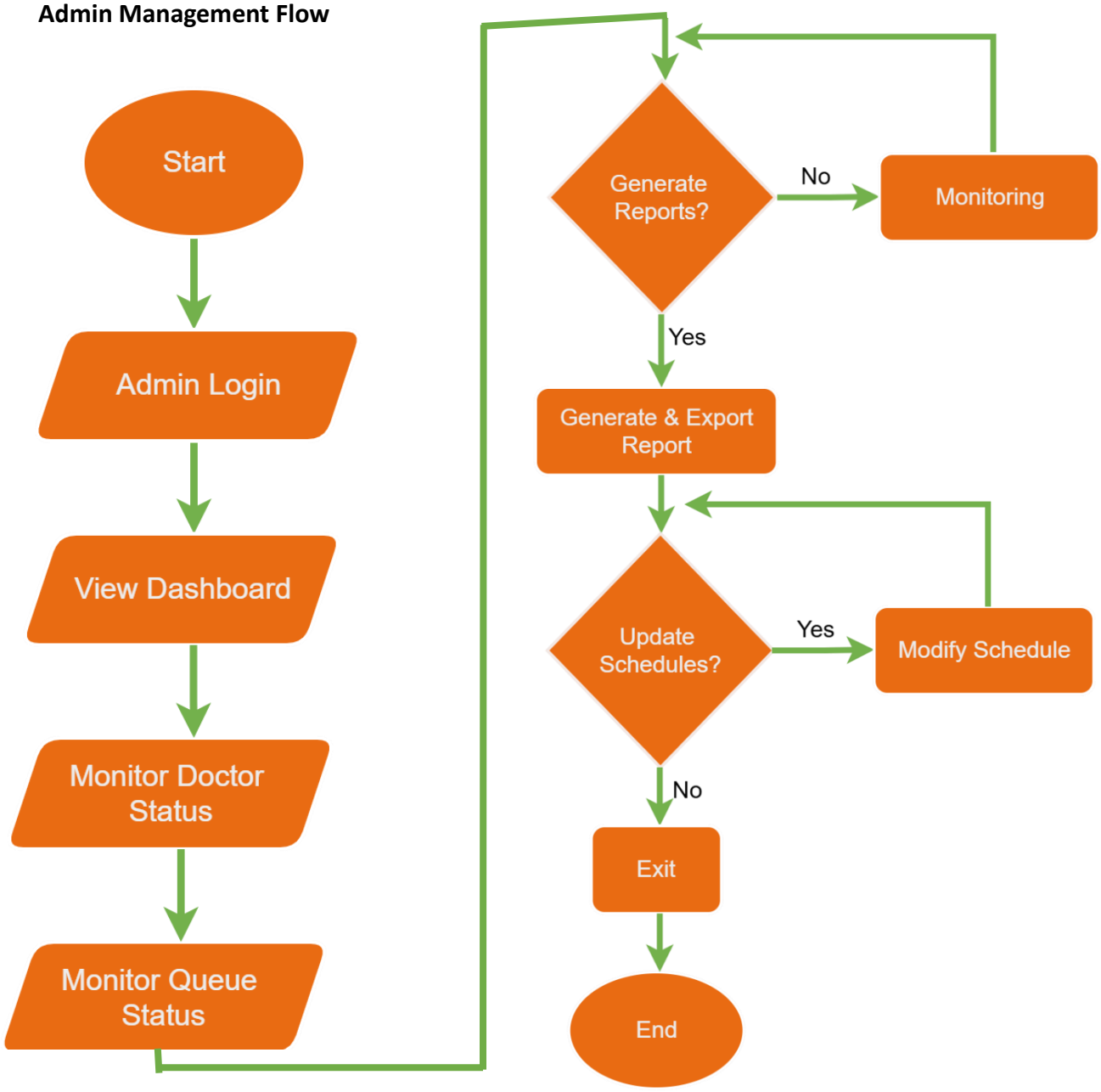
Doctor Status Update Flow



Admin

- Manage Doctors
- Monitor Queue
- Generate Reports
- Configure Notification Rules

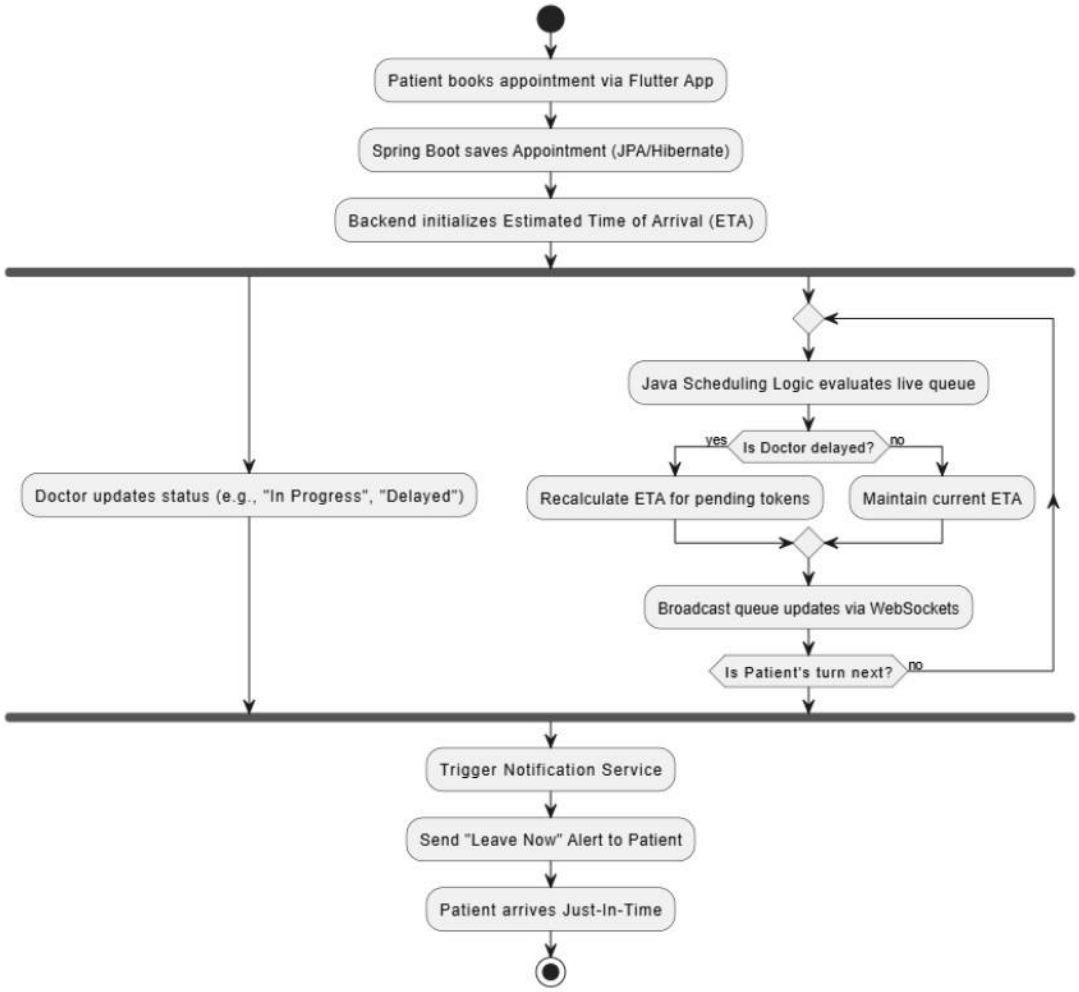
Admin Management Flow



IoT Device

- Detect Room Status
- Send Ready Signal

3.2 ACTIVITY DIAGRAM (Real-Time Scheduling Process)



This diagram maps the step-by-step control flow of the application's core real-time scheduling logic. It demonstrates how the system continuously evaluates the live queue based on doctor progress, dynamically recalculates estimated wait times, and ultimately triggers just in time **Leave Now** alerts to the patients.

3.3 SEQUENCE DIAGRAM (Just-In-Time Alert)

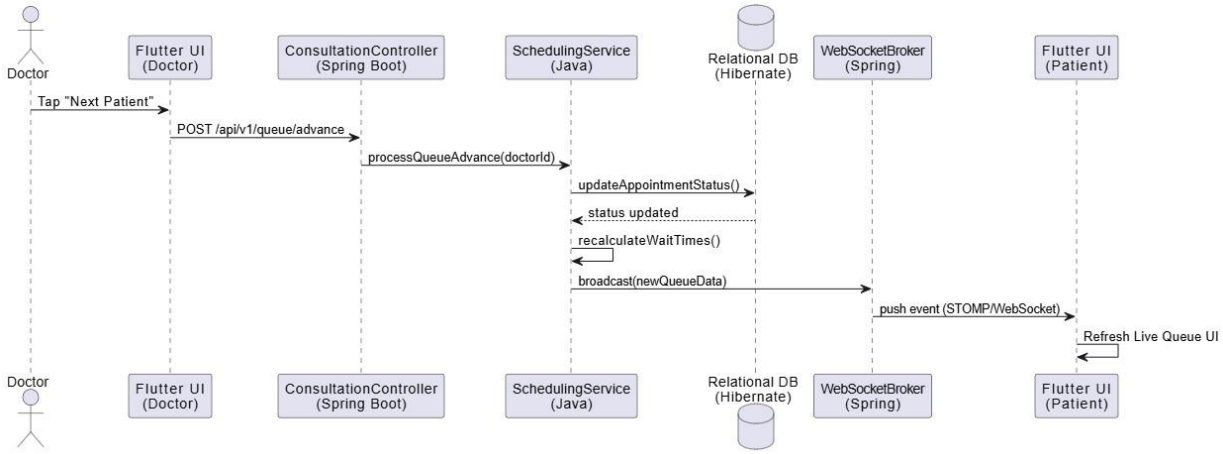
Objects

- Patient App
- Backend Server
- QueueManager
- DoctorInterface
- NotificationService

Sequence

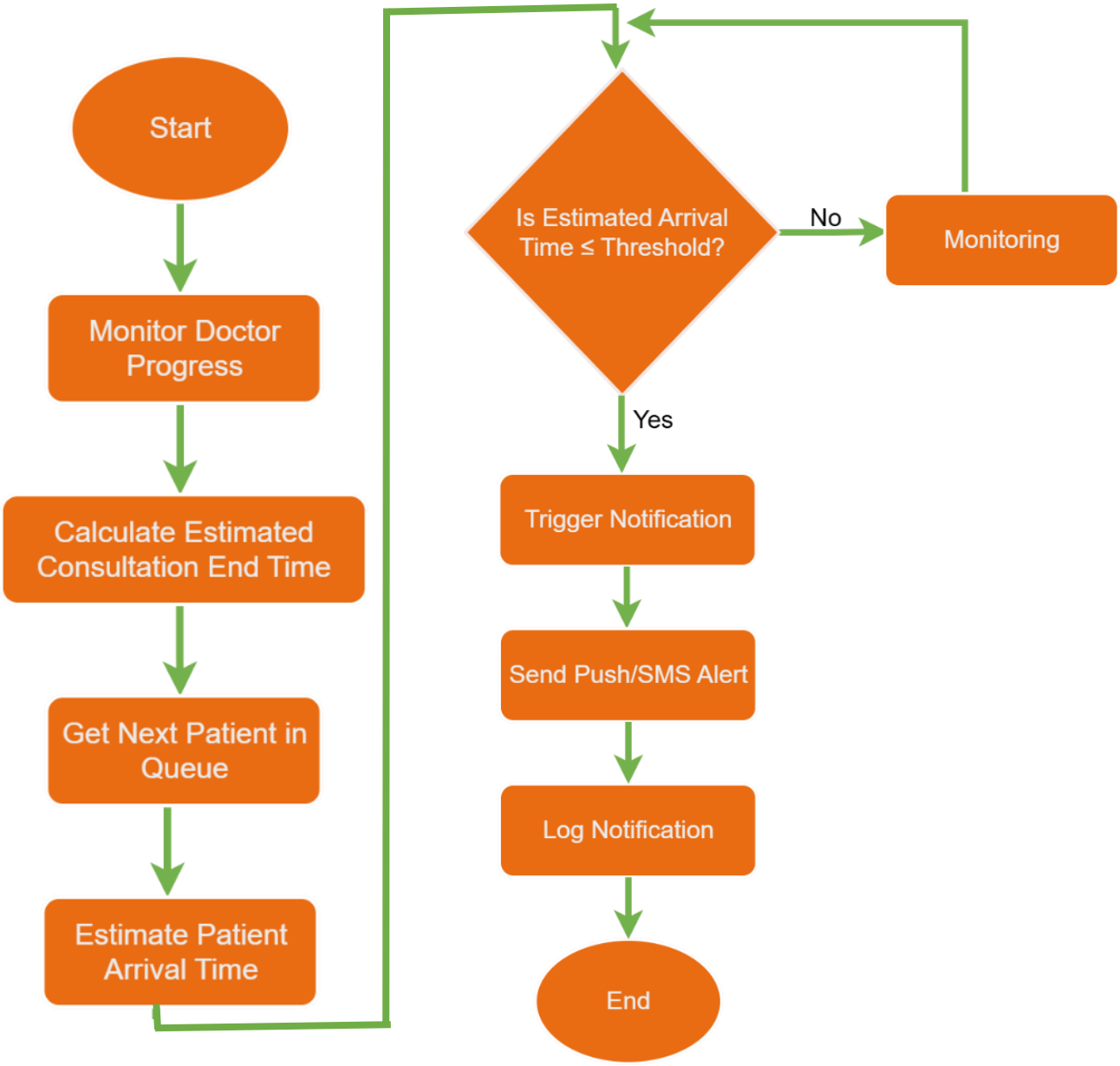
1. Patient App → Backend → Book Appointment
2. Backend → Database → Store appointment
3. Doctor Interface → Backend → Update status
4. Backend → QueueManager → Calculate delay
5. Backend → NotificationService → Trigger alert
6. Notification Service → PatientApp → Send alert

Live Queue Update Flow

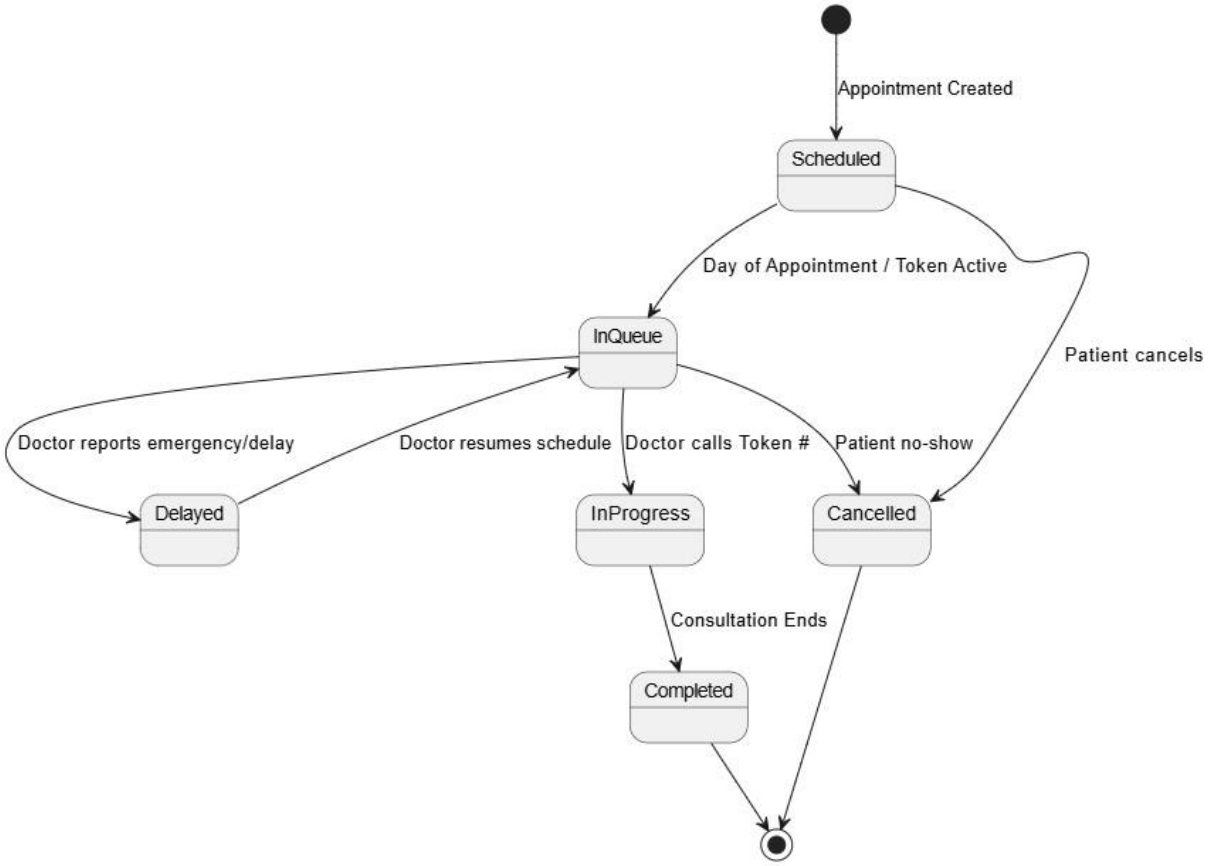


This diagram details the time-ordered interactions across the system's architecture when a doctor updates their consultation status. It traces the communication flow from the Flutter application through the Spring Boot REST controllers, data persistence via Hibernate, and the subsequent WebSocket broadcast that pushes live updates to the patient's interface.

Notification Service Flow



3.4 STATE MACHINE DIAGRAM (Appointment Lifecycle)



This diagram represents the lifecycle and state transitions of an object Appointment within the Spring Boot application. It demonstrates how external events (like a doctor pausing a session or completing a consultation) alter the state of the entity.

4. OBJECT ORIENTED DESIGN (OOD)

4.1 Class Structure (Conceptual)

Class: User

- userID
- name
- email
- login()
- logout()

Class: Patient extends User

- bookAppointment()
- cancelAppointment()
- viewQueue()

Class: Doctor extends User

- updateStatus()
- viewSchedule() Class: Appointment
- appointmentID
- timeSlot
- status
- calculateDelay()

Class: Notification

- notificationID
- message
- send()

Class: QueueManager

- calculateEstimatedTime()
- updateQueue()

Class: IoTDevice

- deviceID
- detectStatus()
- sendSignal()

5. OBJECT ORIENTED PROGRAMMING (OOP)

Implementation in:

- Flutter (Dart OOP)
- SpringBoot (Java)
- Python (ML model via Kaggle)
- ESP32 (Embedded C++)

OOP Principles Applied

Encapsulation → Private fields for patient data

Abstraction → Scheduling logic hidden inside QueueManager

Inheritance → User → Patient, Doctor, Admin

Polymorphism → Different notification types (Push, SMS)

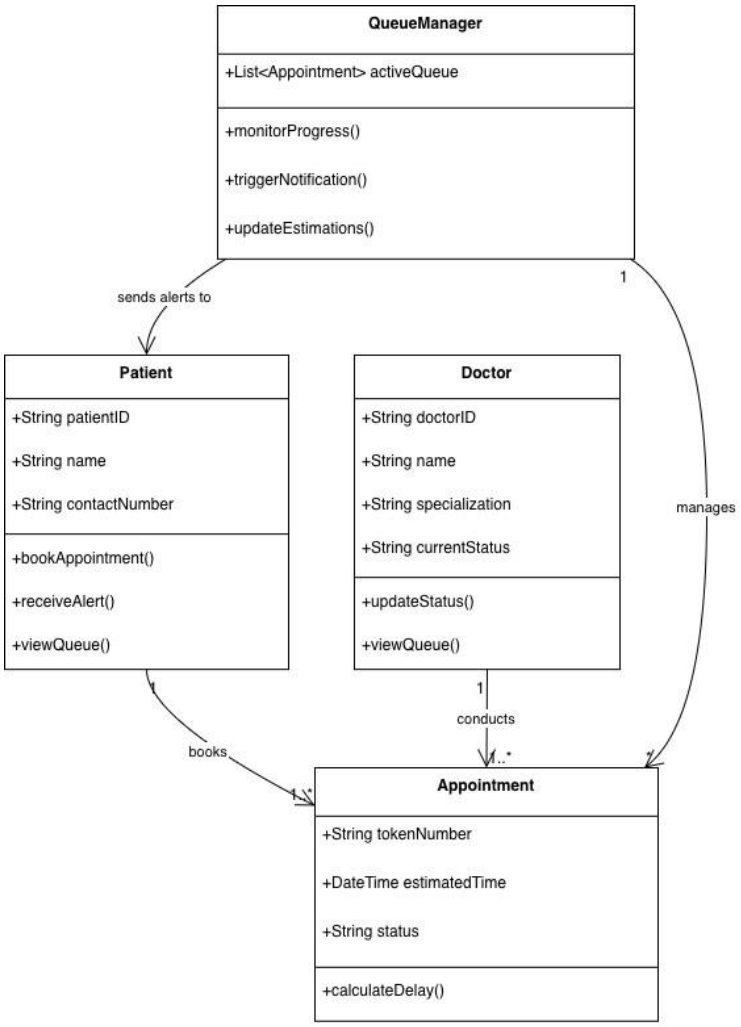
Core System Entities and Object Interactions

To achieve dynamic scheduling, our application represents key parts of the healthcare environment as interacting software objects

- **The Patient:** This entity interacts with the system via a mobile application to book appointments, view available doctors, and receive real-time arrival notifications.
- **The Doctor:** This entity uses a dedicated interface to manage schedules and update their current consultation status (such as started, in-progress, completed, or delayed). Because we are a purely software-based solution, the system relies on the Doctor object's status updates to drive the queue logic.

- **The Appointment:** A distinct object created when a Patient books a time with a Doctor. It stores the queue position, links the Patient and Doctor entities, and is securely stored in the backend.
- **The Queue Manager (Backend System):** The central processing object that continuously monitors the doctor's consultation progress. It calculates dynamic, real time delays and estimates when the next patient should arrive.
- **The Notification:** An object generated by the Queue Manager to send a just in time arrival alert (Please arrive at the hospital in 15 minutes) to the Patient entity.

Conceptual Object Oriented Class Diagram



6. FUTURE ENHANCEMENTS (Advanced OOP Integration)

- AI prediction model (Consultation duration estimation)

- Smart room occupancy detection
- Multi-branch hospital management
- Analytics dashboard
- Blockchain-based medical log security

7. TOOLS USED FOR UML

You must include this section in PDF:

- Draw.io for UML Diagrams
- Figma for UI Design
- Springboot for backend
- Kaggle for ML experimentation
- GitHub for repository management

Main Classes and Relationships

1. User (Abstract Class)

Attributes userID :

String name : String

email : String phone :

String password :

String **Methods**

login() logout()

updateProfile()

Relationship

Parent class of Patient, Doctor, Admin

2. Patient (Inherits User)

Attributes patientID :

String medicalHistory :

String

Methods bookAppointment()

cancelAppointment()

rescheduleAppointment()

viewQueueStatus()

giveFeedback() **Relationships**

1 Patient → 0..* Appointments

1 Patient → 0..* Notifications

3. Doctor (Inherits User)

Attributes doctorID :

String specialization :

String

consultationDuration : int

availabilityStatus : String

Methods

updateConsultationStatus()

markEmergencyDelay()

viewSchedule() **Relationships**

1 Doctor → 0..* Appointments

1 Doctor → 1 Schedule

4. Admin (Inherits User)

Attributes adminID : String

Methods manageDoctors()

manageAppointments()

generateReports()

configureNotificationRules

()

5. Appointment

Attributes appointmentID

: String appointmentDate :

Date appointmentTime :

Time

status : String

estimatedArrivalTime : DateTime

Methods confirmAppointment()

cancelAppointment()

updateStatus() calculateDelay()

Relationships

1 Appointment → 1 Patient

1 Appointment → 1 Doctor

1 Appointment → 0..1 Consultation

6. Consultation Attributes

consultationID : String

startTime : DateTime

endTime : DateTime

consultationStatus : String

Methods

startConsultation() endConsultation()

calculateDuration() **Relationship**

Associated with 1 Appointment

7. Schedule

Attributes scheduleID

: String workingHours

: String breakTimes :

String **Methods**

addTimeSlot()

removeTimeSlot()

updateSchedule()

Relationship

1 Schedule belongs to 1 Doctor

8. QueueManager

Attributes queueID : String

currentQueue : List<Appointment>

Methods

calculateEstimatedTime()

updateQueue()

getQueuePosition() **Relationships**

Manages multiple Appointments Interacts with Doctor and Notification

9. Notification Attributes

notificationID : String

message : String timestamp :

DateTime notificationType :

String (Push/SMS) **Methods**

sendNotification()

scheduleNotification() **Relationship**

Sent to 1 Patient

Triggered by QueueManager

10. IoTDevice (ESP32)

Attributes deviceID : String

deviceStatus : String

Methods detectRoomStatus()

sendReadySignal()

Relationship

Communicates with Backend / QueueManager

@startuml abstract

class User { userID

name email

phone password

login()

logout()

}

```
class Patient {
patientID
medicalHistory
bookAppointment()
cancelAppointment()
}

class Doctor {   doctorID
specialization
consultationDuration
updateConsultationStatus()
}
class Admin {
adminID
generateReports()
}
class Appointment {
appointmentID   appointmentDate
  status
calculateDelay()
}

class Consultation {
consultationID
startTime   endTime
}
```

```
class Schedule {
scheduleID   workingHours
}

class QueueManager {
calculateEstimatedTime()   updateQueue()}

class Notification {
notificationID   message
sendNotification()}
class IoTDevice {
deviceID
sendReadySignal()
}

User <|-- Patient
User <|-- Doctor
User <|-- Admin

Patient "1" -- "0..*" Appointment
Doctor "1" -- "0..*" Appointment
Appointment "1" -- "0..1" Consultation
Doctor "1" -- "1" Schedule
Patient "1" -- "0..*" Notification QueueManager
-- Appointment
```

IoTDevice -- QueueManager

@enduml